押题

1. 概念题/搜索题
2. 逻辑推理/约束满足
3. 概率模型
4. 决策树
5. 人工神经网络

第一讲 绪论

什么是人工智能=》理性agent=》合理的行动

- 像人一样思考：认知建模，认知科学
- 像人一样行动：图灵测试，智能行为测试
- 合理地思考：“思维法则”，逻辑学
- 合理地行动：理性agent，社会行为

教材将人工智能定义为“合理地行动”=》理性agent

agent和理性agent的定义

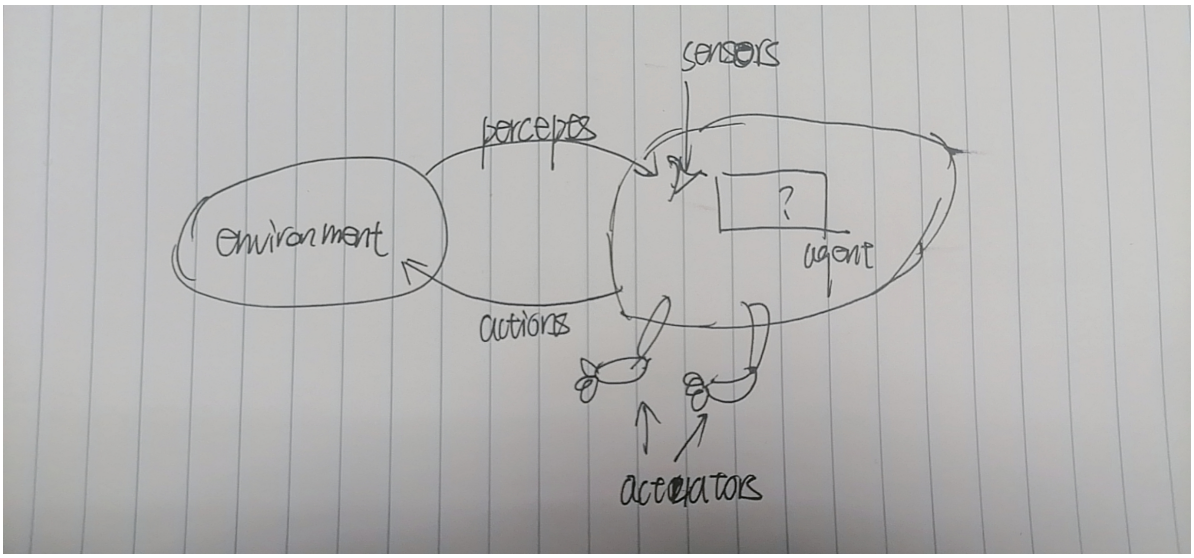
agent：通过传感器感知环境并通过执行器采取行动对所处的环境产生影响的实体。

理性agent：能够实现**最佳期望结果**而行动的agent

第二讲 智能Agent

agent与环境

- agent将任何感知历史数据映射为行为
- agent程序通过在物理系统上运行来实现agent函数
- agent = 体系结构 + 程序



理性agent

理性agent：对于每一可能的感知数据序列，一个理性的Agent应该采取一个行为**以到达最大的性能**

理性判断的4个因素：

- 性能度量
- 先验知识
- 可以完成的行动
- 截止到此刻的感知序列

个人理解：代价函数，训练出的模型参数，可能的输出结果，历史数据

任务环境

任务环境包括：

- 性能Performance
- 环境Environment
- Agent的执行器Actuators
- 传感器Sensors

缩写为PEAS

对于每一个智能Agent，必须说明其PEAS参数

环境的性质

- 完全可观察的 vs. 部分可观察的

一个agent的传感器在每个时间点上都能获取环境的完整/部分状态

- 确定的 vs.随机的

环境的下一个状态是否完全取决于当前状态和Agent执行的动作

- 片段式的 vs. 延续式的

对于片段式：Agent的经历被分成一个个原子片段，在每个片段中Agent感知信息并完成单个行动，下一个片段不依赖于以前的片段

- 静态的 vs. 动态的

环境在Agent计算的时候不会变化(vs.会变化)

半动态的:环境本身不变化但agent的性能评价随时间变化

- 离散的 vs. 连续的

环境的状态，时间的处理方式以及Agent的感知信息和行动都有离散/连续之分

- 单agent vs. 多agent

单agent独自运行

多agent同时运行

agent函数和程序

- 一个agent用agent函数来表示
- agent函数将感知数据序列映射为行为
- agent程序
    - 以传感器得到的当前感知信息为输入
    - 以执行器的行动为输出
    - 仅仅以当前感知为输入而不是以整个历史感知为输入

agent的四种类型！！！

- 简单反射agent：基于当前感知选择行动，不关注感知历史
- 基于模型反射的agent：agent根据感知历史维护内部状态，agent随时更新内部状态信息
- 基于目标的agent：除了根据感知信息外，还要根据目标信息来选择行动。效率比较低，需要推理
- 基于效用的agent：当达到目标行为有很多种时，需要考虑效率。

  =》环境是部分科观察和随机的，不确定下的决策过程可以通过基于效用的agent来实现。

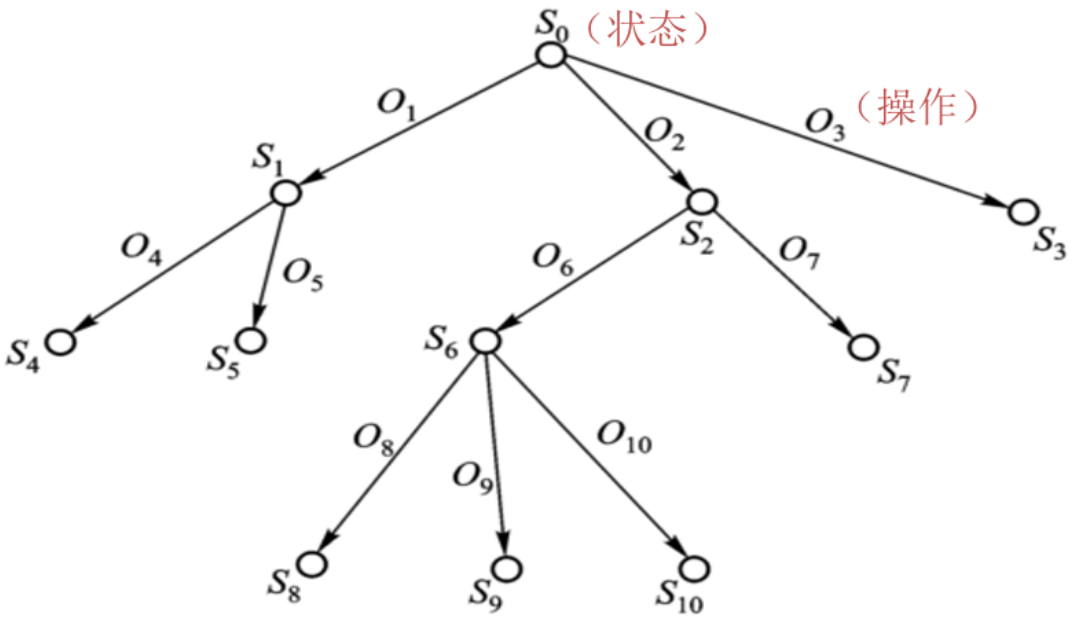
  效用的作用：

  - 多目标相冲突时
  - 多目标在不确定环境中
  - 一个目标有多种行为可以达到时

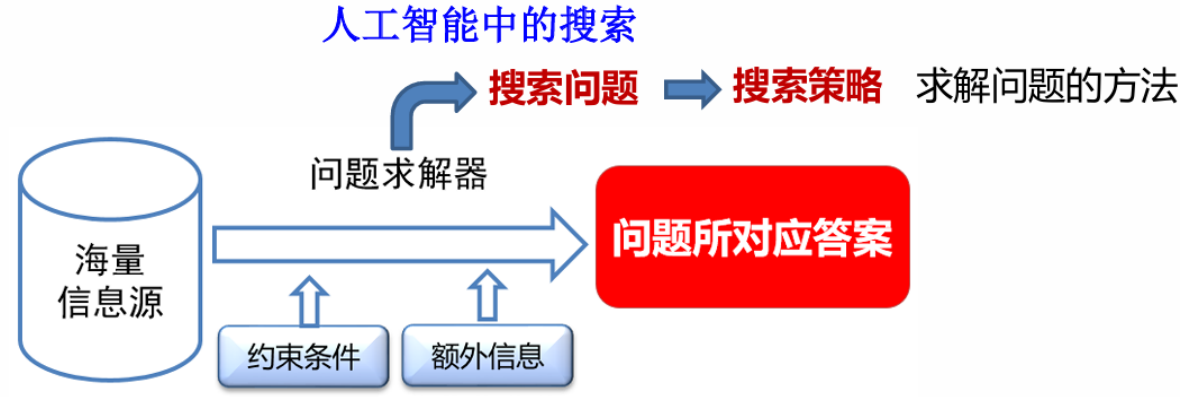
第三讲 无信息的搜索

状态空间图

状态空间有向图



问题求解agent



搜索——搜索算法本质上是一个程序，旨在找到到达目 标的最佳或最短路径。



树搜索

基本思路：从初始状态/已知状态开始，通过行动不断地探索**其他状态**直到找到目标状态(成功)或者没有行动可执行为止(失败)

树表示：

- 根节点：初始状态
- 连线：行动
- 结点：状态空间中的状态

搜索策略、完备性、最优性、两个复杂度

搜索策略是指搜索树结点选择的搜索顺序

- FIFO队列
- LIFO队列
- 优先级队列
- 哈希表：快速有效检测重复状态

性能评价标准

- 完备性：如果问题存在解，算法即可找到解
- 最优性：找到的解是最优解
- 时间复杂度：花费的时间，由搜索过程中产生的结点数目来度量
- 空间复杂度：花费的内存，由内存中存储的最多结点数量来度量

无信息搜索策略

无信息搜索

- 除了问题定义中提供的状态信息外没有任何附加信息
- 算法只能区分状态是不是目标状态
- **无法比较非目标状态的好坏**

树搜索和图搜索（这个有点违反一般理解）

树搜索和图搜索都是在图上搜索

不过树搜索不会标记已经扩展过的节点，所以可能会扩展一个节点多次，但是不会走回头路

图搜索则是会标记已经扩展过的节点，所以一定不会访问一个节点多次

策略

分支因子b

目标节点的深度d

最深深度m

评价函数f(n)=g(n)+h(n)

代价函数g(n)

启发函数h(n)

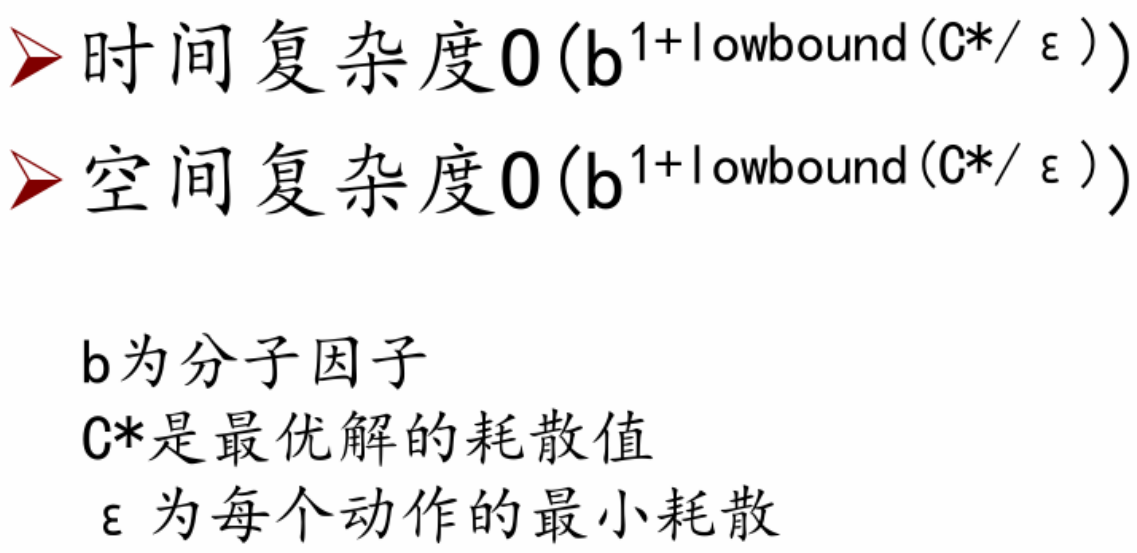
最优深度C*

宽度优先搜索

- 先扩展根结点，再扩展根结点的所有后继， 然后再扩展它们的后继
- 实现：FIFO队列
- 完备的：如果最浅的目标节点处于一个有限深度d，bfs在扩展完比它浅的所有节点（假设分支因子b是有限的）之后最终一定能找到目标节点
- 非最优的，最浅的目标节点不一定就是最优的目标节点。（如果路径代价是基于节点深度的非递减函数，bfs是最优的。最常见的就是当所有行动要花费相同的代价。
- 时间复杂度是b^d
- 空间复杂度是b^d，因为搜索队列中需要维护所有叶子节点，叶子节点最多的时候就是到搜索树底层时为b^d

一致代价搜索

- 扩展未扩展结点中代价g(n)最小的
- 非完备的，对解路径的步数并不关心，只关心路径总代价。所以，如果存在0代价行动就可能陷入死循环；但如果每一步代价都大于等于某个小的正值常数ε，那就是完备的
- 显然是最优的，每一步是非负的且随节点增加路径不会变短
- 两个复杂度：



=》路径的深度不能用d而是取决于最优深度C*

深度优先搜索

- 首先扩展最深的为扩展结点
- 避免重复状态和冗余路径的图搜索，在有限的空间是完备的，毕竟顶多扩展至所有节点；树搜索不完备，存在有限空间的死循环和冗余路径。
- 不是最优的，优先访问左子树但最优路径可能在右子树
- 时间复杂度是b^d，毕竟顶多扩展至所有节点
- 空间复杂度是bm，因为只用维护root到当前的路径，所以最大也就bm，其中m是最深深度

深度受限搜索

- 将dps搜索的深度设置一个界限l
- 如果界限l<d，不是完备的，否则是完备的
- 就算l>d，也不一定是最优的，理由和bfs的一样：最浅的目标节点不一定就是最优的目标节点，但如果路径代价是基于节点深度的非递减函数，bfs是最优的。最常见的就是当所有行动要花费相同的代价。
- 时间复杂度b^l
- 空间复杂度bl，这个是dfs的优势

迭代加深的深度优先搜索

- 不断的增加l直到找到目标
- 和bfs一样，当分支因子b是有限的是完备的
- 和bfs一样，当路径代价是节点深度的非递减函数时是最优的
- 空间复杂度bd，这个是dfs的优势
- 时间复杂度b^d

ps：也许迭代加深的dfs看起来比较浪费，因为状态被多次重复生成。但事实上代价并不是多大。原因是在分支因子相同（或者近似）的搜索数中，绝大多数的结点都在底层，所以上层的结点重复生成多次影响不大。

第四讲 有信息的搜索

贪婪最佳优先搜索

- 扩展未扩展结点中启发函数h(n)最小的
- 和dfs一样，一般不是完备的，除非是有限状态空间的图搜索
- 非最优的，毕竟是“贪婪”的——只关注h(x)忽略g(x)
- 时间复杂度都是b^m，m是搜索空间的最大深度
- 空间复杂度都是b^m，m是搜索空间的最大深度，显然在搜索的时候需要维护一搜索过的所有结点

A* 搜索

- 扩展未扩展结点中评估函数f(x)=启发函数h(x)+代价函数g(x)最小的
- 如果代价≤最优深度C*的节点是有穷的，就是完备的
- 如果h(x)是可采纳，那么树搜索是最优的；如果h(x)是一致的，那么图搜索是最优的
- 复杂度则严重依赖于状态空间的情况

可采纳的和一致性启发

启发函数h(n)是可采纳的条件：如果对于每个结点n, h(n)=<h*(n)，其中h*(n) 是到达目标结点的真实代价=》**可采纳启发函数绝不会高估到达目标结点的代价，因此它是最优的**

启发函数h(n)是一致的条件： 对于任意一个结点n，以及n的行为a产生的后继结点n',满足如下公式：
h(n) ≤ c(n,a,n') + h(n')

- **所有一致的启发式函数都是可采纳的**：如果一个启发式函数是一致的，那么它自然是可采纳的，因为一致性的定义已经隐含了它不会高估任何节点到目标节点的实际最低成本。除非h（G）>0，其中G为目标节点
- **并非所有可采纳的启发式函数都是一致的**：可采纳性仅要求启发式函数不高估到目标节点的成本，而不要求启发式函数之间的任何特定关系满足一致性条件。

第五讲 超越经典的搜索

搜索算法

面临挑战

（1）求解问题系统不可能知道与实际问题有关的全部信息， 因而无法知道该问题的全部状态空间，也不可能用一套算法来求解所有的问题。

（2）有些问题在理论上虽然存在着求解算法，但是在工程实 践中，这些算法不是效率太低，就是根本无法实现。

搜索算法分类

- 基本搜索算法：宽度和深度优先搜索法
- 启发式搜索算法：A*搜索、爬山法
- 博弈搜索：极小极大(Minimax)算法
- 智能优化算法：模拟退火、遗传算法、粒子群优化算法 **智能优化算法属于启发式算法。**

智能优化算法-启发式优化算法

“启发式”（Heuristic）：在状态空间搜索中，启发式被定义成一系列操作算子，并能从状态空间中选择最有希望到达问题解的路径。

启发式策略：利用与问题有关的启发信息进行搜索。

智能优化算法又叫启发式优化算法，是受自然界的现象、规律或机制的启发而提出的算法，具有全局优化性能、通用性强且适合于并行处理。

目前大约有90多种智能优化算法，常用的智能优化算法有：遗传算法 、模拟退火算法、禁忌搜索算法、粒子群算法、蚁群算法等。

爬山算法

局部搜索算法：从单个当前结点出发，通常只移动到它的邻近状态而**不保留搜索路径**

爬山算法：一种**局部**择优的人工智能算法，采用启发式方法， 是对深度优先搜索的一种改进，算法每次从当前解的临近解空间中选择一个最优解作为当前解， 直到达到一个局部最优解。

优点：避免遍历，通过启发选择部分节点，从而 达到提高效率的目的。

缺点：

1）局部最大：某个节点比周围任何一个邻居都高，但 是它却不是整个问题的最高 点。

2）高地：也称为平顶，搜 索一旦到达高地，就无法确 定搜索最佳方向，会产生随机走动，使得搜索效率降低。

3）山脊：搜索可能会在山 脊的两面来回震荡，前进步伐很小。

爬山算法每次从当前解的临近解空间中选择一个最优解作为当前解，直到达到一个局部最优解。

爬山算法实现很简单，其主要缺点是会陷入局部最优解， 而不一定能搜索到全局最优解。

和一致代价的区别和联系：

区别：一致代价拥有全局视野，找到的一定是全局最优解，具有最优性。而爬山只看得到局部状态，没有最优性

联系：都是选择代价g(x)最小的继续扩展

模拟退火

爬山法不完备，随机法效率低，考虑结合两者产生了模拟退火搜索

基本思想： 模拟退火其实也是一种贪心算法， 在搜索过程引入了随机因素。 在迭代更新可行解时，以一定的概率来接受一个比当前解要差的 解，因此有可能会跳出局部最优解，达到全局的最优解。 随着时间的推移，允许算法向坏的方向移动的概率逐步下降。

如果温度下降得足够的慢，那么模拟退火算法找 到一个全局最优值的概率接近于1

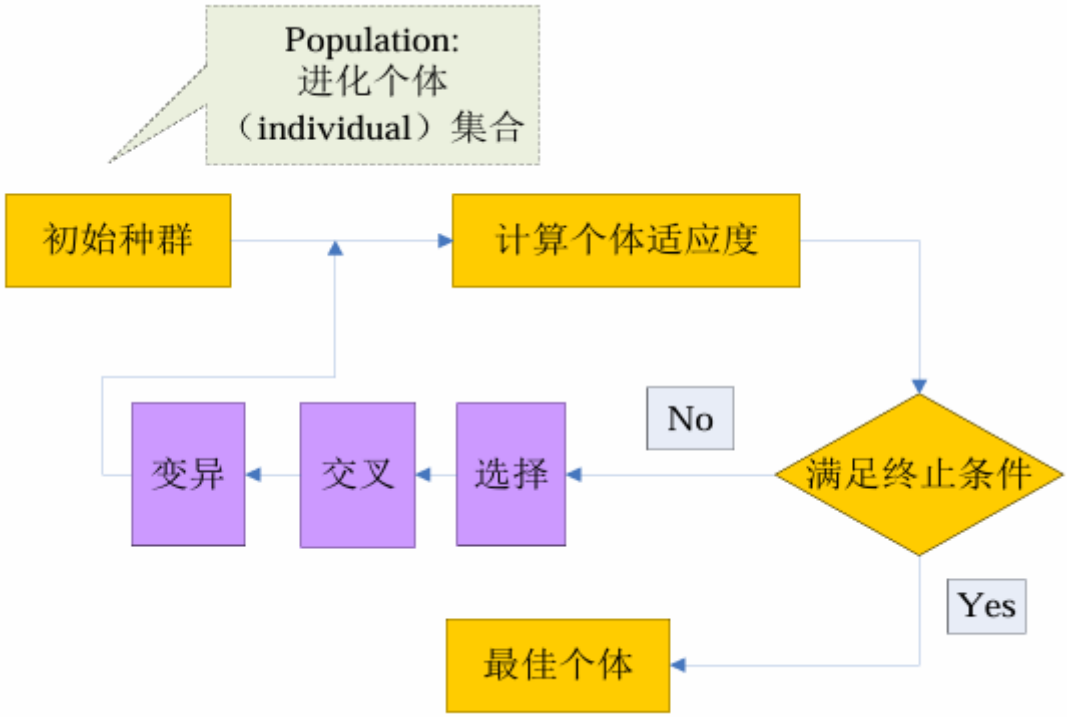
遗传算法

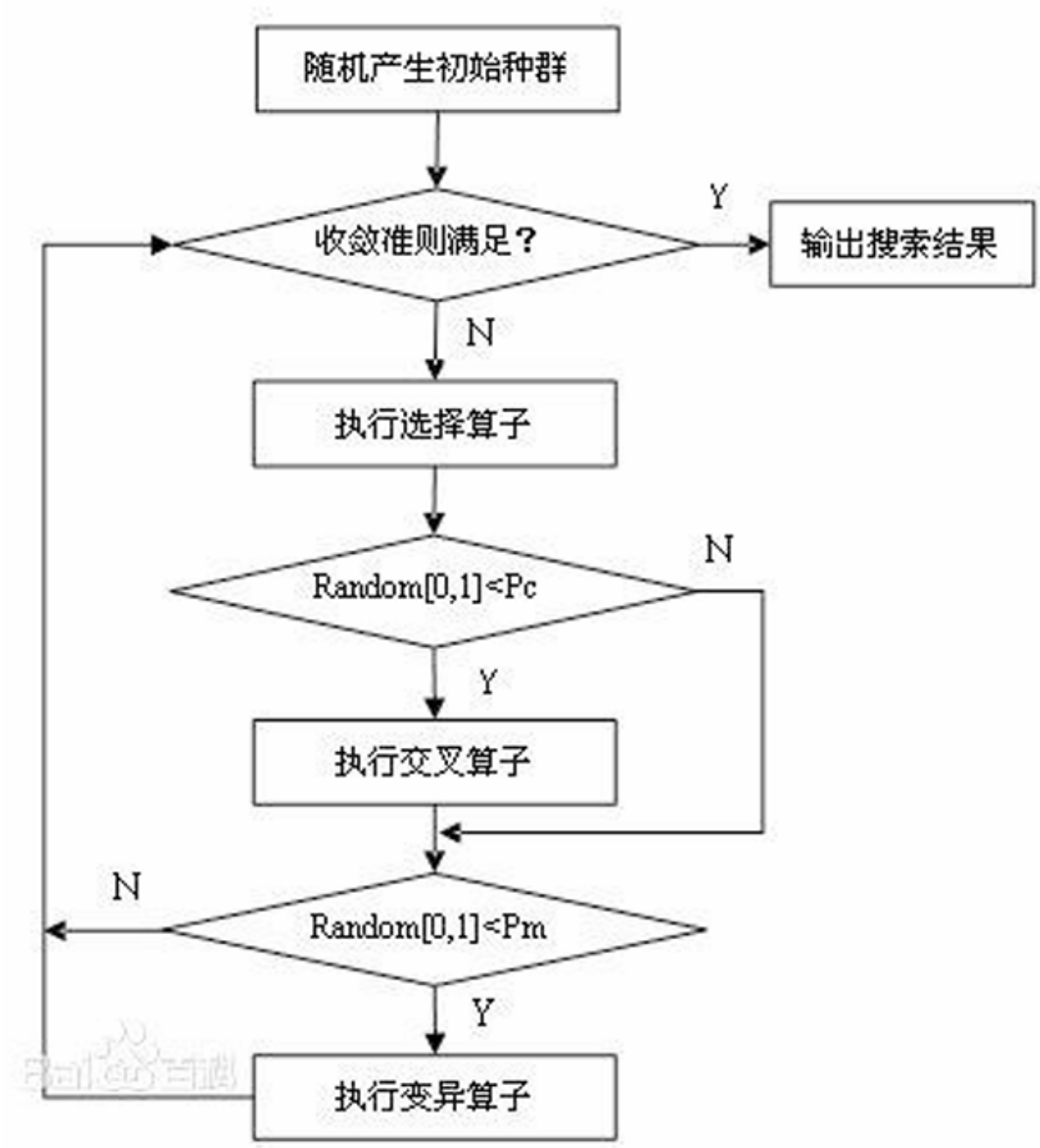
核心思想和流程图

（1）自然界的进化准则

（2）生物的遗传学原理

遗传算法是模仿生物遗传学和自然选择机理，遗传算法以“适者生存”的生物进化理论为基础的 一种智能优化算法。最适合自然环境的群体往往 产生了更大的后代群体。通过**选择、交叉和变异**这三种操作实现种群进化的计算模式。


Population:
进化个体
（individual）集合
初始种群
计算个体适应度
满足终止条件
No
选择
交叉
变异
Yes
最佳个体



- 选择操作一定会做，但交叉和变异是概率去做

生物遗传对应的算法遗传概念

生物遗传概念	遗传算法的概念
适者生存	目标值优的解被选择的可能性大
个体（Individual）	解
染色体（Chromosome）	解的编码（字符串、向量等）
基因（Gene）	解的编码中每一分量
适应性（Fitness）	适应度函数值
群体（Population）	根据适应度值选定的一组解（解的个数为群体的规模）
婚配（Marry）	交叉（Crossover）选择两个染色体进行交 叉产生一组新的染色体的过程
变异（Mutation）	编码的某一分量发生变化的过程

无观察信息的搜索

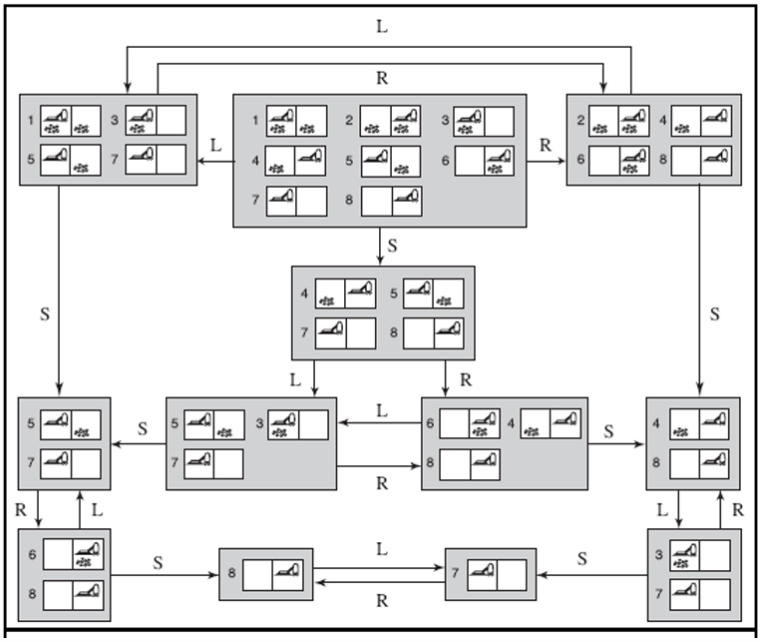
- **Agent 感知不到任何信息**，称无传感问题，也称相容问题
- 无传感问题是可解的还是无解的？ **可解！**

定义

- 无观察信息问题P的定义
  - 信念状态：包含物理状态中每个可能的集合，假定N个物理状态，最多有2^N个信念状态
  - 初始状态：所有物理状态的集合
  - 行动： $ACTION(b) = \bigcup_{s \in b} ACTION_P(s)$
  - 转移模型：
    - 对于确定行动 $b' = \bigcup_{s \in b} RESULT_P(b, a)$
    - 对于不确定行动$b' = RESULT(b, a) = \{s' : s' = RESULT_P(s, a) 且 s \in b\}$
  - 目标测试：信念状态中的所有物理状态都满足目标状态
  - 路径开销：假定所有状态下一个行动的开销相同

eg：在真空吸尘器这个例子中，左/右边有无灰尘，车在左右共有2^3=8种物理状态，而这8种状态选择其中几个打包的可能性有2^8=256种信念状态

➢256个可能的信念状态只有12个可达；
➢初始状态出发的行动序列{S, L, S}与{R, L, S}达到相同的信念状态{5, 7}
➢如果一个行动序列是信念状态b的解，那么它也是b的任何子集的解



（其实有点像项集）

部分可观察信息的搜索

=》**就是用观察到的信息去剪掉信念状态里的不可能的状态**，或者说，约束满足？

部分可观察环境中的问题求解Agent

- 形式化，搜索算法，执行解行动
- 解是一个条件规划不是一个序列 if-then-else
- Agent在完成行动和接收感知信息时维护自身的信念状态

第六讲 对抗搜索

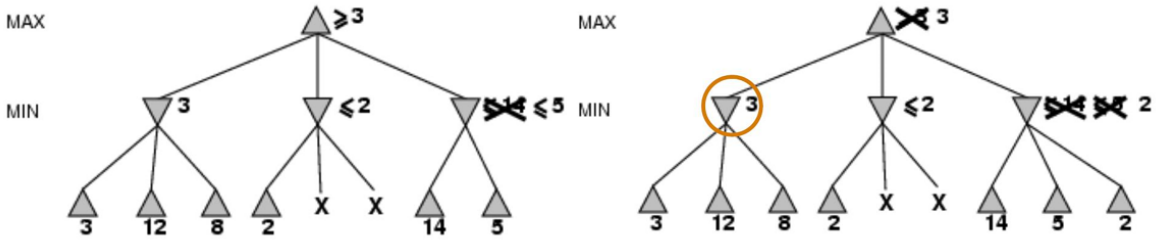
博弈

竞争环境中多个Agent之间的目标是有冲突的， 称为**对抗搜索问题**，也称为**博弈（Games）**

α-β剪枝

即叶节点[alpha,beta]，这个剪枝是横跨了祖孙三代的：

其中，alpha是下限，是儿子们都是MIN时从一个MIN儿子传到另一个MIN儿子的，儿子们如果发现任意孙节点比alpha还小就不用搜索了，反正MAX爷节点不会要剩余孙节点搜出来的结果：



原理：

如图，在搜索完左边字数最小值为3

搜索中间子树第一个叶节点为2，显然：因为取最小值，就算搜完剩下两个叶节点该子树结果也会≤2，而根节点因为左子树的结果≥3（alpha=3），所以就没必要搜索剩下两个叶节点了

不完美的实时决策

当有资源限制时考虑的解决方案：

- 限制搜索的时间和深度
- 评估当前状态的效用：比如线性评估。（ps：评估函数只适应于静态棋局，即不会很快出现大摇摆的棋局）
- 前向剪枝：无需考虑直接剪枝一些子结点
- 柱搜索：每一层只考虑最好的n步棋=》可能导致最佳的行棋被剪掉（Probcut算法：使用先验的统计信息在一定程度上保护最佳行棋不被剪枝掉，首先浅层搜索计算结点的v值，再根据经验来估 计深度d上的值是否在(α,β)范围外）
- 搜素与查表

随机博弈

许多博弈存在不确定性的随机因素，如掷骰子， 我们称为随机博弈，如西洋双陆棋

部分可观察的博弈

eg：

军棋：棋子可以移动但对方看不见棋子是什么，使用信念状态

牌类：随机部分可观察，需要概率推算来制定决策

第七讲 约束满足问题

约束满足问题的定义

约束满足问题（CSP，Constraint Satisfaction Problem）由一个变量集合和一个约束集合组成。

每个变量有自己的值域，当**每个变量都有 自己的赋值同时满足所有关于变量的约束**时， 问题就得到了解决，这类问题就叫做约束满 足问题。

一个CSP问题的定义包含三个成分：

- 变量的集合X={X1,...,Xn}
- 值域的集合D={D1,...,Dn}
- 约束条件集合C={C1,...,Cn}

一元约束：只涉及一个变量 Eg., SA!=green

二元约束：涉及两个变量 Eg.,SA!=WA

高阶约束：涉及3个或者3个以上变量

全局约束：变量个数任意的约束称为全局约束

约束满足问题的推理 （约束传播？）

约束传播：使用约束来减小一个变量的合法取值范围，从而影响到跟此变量有约束关系的另一变量的取值， 如此进行。

结点相容：单个变量值域中的所有取值满足一元约束

弧相容：某变量值域中的所有取值满足该 变量的所有二元约束

通用弧相容：Xi相对于某n元约束是相容的

路径相容：两个变量集合{Xi,Xj}对于第三个变量Xm是相容的，指{Xi,Xj}的每一个相容赋值{Xi=a,Xj=b}, Xm都有合适的取值同时使得{Xi,Xm},{Xm,Xj}是相容的

K相容：如果对于任何k-1个变量的赋值， 第K个变量总能被赋予一个和前面k-1个变量相容的值（经典场景：一步步给图图上颜色）（如果一个图是k相容的，k-1相容的，...,直 1相容的，那么此CSP是强k相容的。）

约束满足问题的回溯搜索

其实就是dfs的咯

考虑提高算法效率的策略：

- 选择变量赋值的顺序
  - 最受约束变量：优先选择最少剩余值的变量进行赋值
  - 约束最多变量：优先选择最能约束其他变量的变量进行赋值
- 选择值域中赋值的顺序
  - 最少约束值方法： 选择使得剩余变量赋值空间更大的值
- 更早地检测不可避免的失败
  - 前向检验：追踪未赋值变量的剩余合法赋值，当任何一个剩余变量没有合法赋值时搜索终止。（前向检测从赋值变量向未赋值变量传播信息， 但不能对失败提供早期检测）
  - 智能回溯

约束传播中的问题结构

独立子问题

独立的子问题：不相连的结构

- e.g. Tasmania 与其他地区不相连

独立性子问题通过寻找连通子图来确定

- Si是CSPi的解， ∪iSi是∪iCSPi的解
- 总的工作量从O(dn)减少到O(dcn/c),c是子问题 变量数

拓扑结构

应用拓扑排序来求解树结构的CSP

- 任意选择一个变量为树的根，选择变量顺序使 每个变量在树中出现在父结点之后
- n个结点n-1条弧，O(n)步将此图改造成直接弧 相容,每一步比较两个变量的d个可能取值，所 以总时间为O(nd^2)

约束图到树

基于删除结点的方法

- 从CSP的变量中选择子集S, 使得约束图在删除S后成为 一棵树，S称为环割集
- 对于满足S所有约束的每个可能赋值
  1. 从CSP剩余变量的值域中删 除与S的赋值不相容的值
  2. 如果去掉S后的剩余CSP有解，把解和S的赋值一起返回

基于合并结点的方法

- 把约束图分解成相关联的子问题
- 对每个子问题单独求解，再把结果合并起来

树分解的四个条件

- 每个变量至少出现在一个子问题中
- 每个约束至少出现在一个子问题中
- 如果一个变量出现在两个子问题中， 那么它必须出现在连接在这两个子问题的路径上的所有子问题中
- 如果一个约束出现在两个子问题中， 那么它必须出现在连接在这两个子问题的路径上的所有子问题中

第八讲 逻辑agent

基于知识的Agent

域独立算法=》推理引擎

域相关内容=》知识库（KB）：一种正式语言的语句集合（行为Tell添加，行为Ask查询）

公理：预先给定的而不是推导的语句

agent要求要能够：

- 表示状态，行动
- 引入新的感知信息
- 更新世界的表示
- 推导世界的潜在属性
- 推导正确的行为

逻辑、模型、蕴含、推理

逻辑：一种用来表示信息的正式语言，该语言能够推导出一定的结论（约束传播？）

语法：用来定义语言中句子的规范准则（约束？）

语义：用来定义句子的含义，也定义了每个句子在可能世界中的真值（值域？）

模型：来代替"可能世界"（对真实世界建模？）

逻辑蕴含：某语句在逻辑上跟随另一语句（α|=β表示在使α为真的每个模型中，β也为真）

- KB|=α当且仅当M(KB)真包含M(α)

推理：KB ├iα 表示推理算法i可以根据KB推导出α

- 可靠性：如果当KB ├i α, KB|=α也是真的，那 么i是可靠的
- 完备性：对于任何一个KB|=α, 推导算法i能够 得到KB ├i α ， 那么i是完备的

蕴含就像**干草堆(KB)中的一根针(α),推导 算法的目的就是找到这根针**

命题逻辑

定义

命题逻辑：应用一套形式化规则对以符号表示的描述性陈述进行推理的系统

语法：用来定义合法的语句

原子语句：由单个命题词组成，每个命题次代表一个为真或者假的命题

命题连接词：与^，或v，非¬，条件=>，双向条件<=>（顺序优先级高到低：¬，^，v，=>，<=>）

逻辑等价：如果两个语句α，β在同样的模型集合中为真，记为α≡β。α≡β当且仅当α|=β且β|=α

有效的：语句在所有模型中都为真

可满足的：语句在某一个模型中为真

不可满足的：语句在所有模型中都为假

推导证明

证明分为两类：

- 推理规则（就离散那一套东西）
    - 从旧的语句中产生新的语句
    - 证明=一系列推理规则的应用
    - 通常需要将语句转成范式
- 模型检查
    - 真值表枚举（穷举法）
    - 在模型空间中进行启发式搜索（A*）

推理规则

- 假言推理规则/分离规则（Modus Ponens，简称M.P.）：肯定前件，就要肯定后件；否定后件，就要否定前件（逆否问题）。
- 消去合取词规则/与消解（And-Elimination，简称A.E.）：A^B为真，那A为真，B为真
- 与导入：上述过程反过来
- 双重否定：双重否定等于没否
- 单项消解/单项归结：αVβ^¬β可推出α
- 消解/归结：（αVβ）^（¬βVγ）可推出αVγ

合取范式CNF

CNF：e.g. （AVB）^（CVD）

转换成CNF的步骤：

转换成合取范式

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

1. 消去等价词

$$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

2. 消去蕴含词

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

3. 用De Morgan定律把否定词放在字符前面

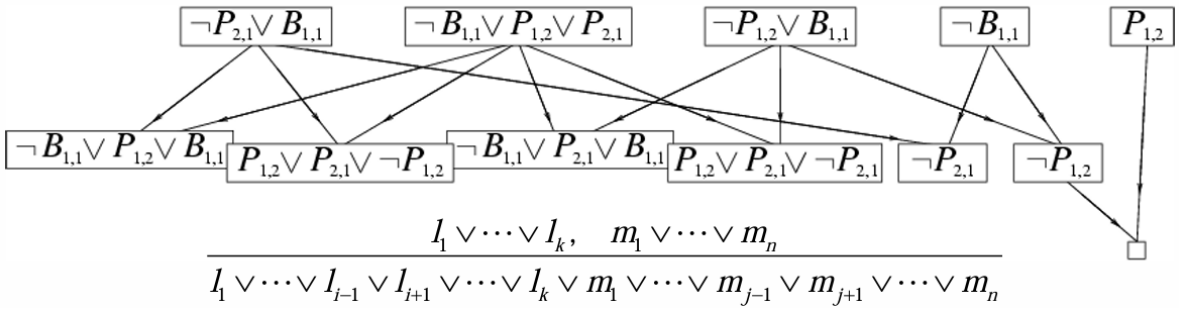
$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$$

4. 用分配律改变合取和析取符号的位置

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

归结算法（反证法）

- 为了证明KB=|α，需要证明(KB∧¬α)是不可满足的
- 例子： Wumpus
  - KB=$(B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1}$　　$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
  - α = $\neg P_{1,2}$　　$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$



$$\frac{l_1 \vee \cdots \vee l_k, \quad m_1 \vee \cdots \vee m_n}{l_1 \vee \cdots \vee l_{i-1} \vee l_{i+1} \vee \cdots \vee l_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n}$$

DPLL算法

是一种完备的、以回溯为基础的算法，用于解决在 合取范式（CNF）中命题逻辑的布尔可满足性问题。（实际上就是深搜CNF的取值）

比枚举算法的改进：

- 及早终止算法
  - 一个子句是真如果其中任意一个文字为真

    - 一个子句是假如果所有的文字都为假
- 纯符号启发式
    - 纯符号：某个文字在所有的子句中都是正或者负
    - (A∨ ¬ B), (¬ C∨ ¬ B), (C∨ A), A,B为纯符号，C 不是
    - 找到纯符号能够通过对纯符号的赋值使句子为真
- 单元子句启发式
    - 单元子句：子句中除了某个文字外其他文字都是假的子句
    - 单元子句中的文字必须为真 e.g. 已知B为真，(¬ C∨ ¬ B)变为单位子句¬C

WalkSAT算法

不完整的局部搜索算法

爬山法、模拟退火。。。

算法的思想：每次迭代时选择一个未得到 满足的子句并从该子句中选择一个符号对 其进行翻转，翻转符号的选择方法：

1）最小化新状态下未满足语句的数量

2）随机挑选符号

评估函数：最小化不满足的子句数量

第九讲 一阶逻辑（谓词逻辑）

一阶逻辑与命题逻辑的不足

命题逻辑的局限性 在命题逻辑中，每个陈述句是最基本的单位

命题逻辑中，不能表达局部与整体、 一般与个别的关系。引入更加强大的逻辑表示方法——谓词逻辑

一阶逻辑的语法和语义：谓词，量词，函词

谓词

在谓词逻辑中，将原子命题进一步细化，分解出 个体、谓词和量词，来表达个体与总体的内在联 系和数量关系，这就是谓词逻辑研究内容。

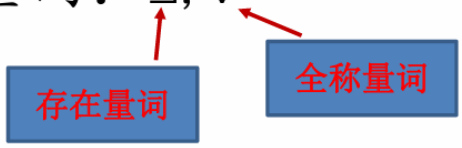
一阶逻辑的语法：基本元素

- 常量符号：Hunan，University，…
- 谓词符号：Brother，＞，...
- 函数/函词：Sqrt，LeftLegOf，...
- 变量：x，y，a，b，…
- 连接符：∧，∨，¬，…
- 量词：∃，∀

个体

谓词（predicate）

量词（quantifier）



一阶逻辑的语法：基本元素　谓词与个体

- $P(x)$ 表示：$x < x^2$

- P是谓词，一般用大写字母P, Q, R等来表示谓词。
  $P(x)$描述了是否存在一个数，这个数小于自身平方。

- x是个体词，x被称为变量。x的取值叫个体常项。
  比如，$P(0.1)$ 和$P(0.02)$使得谓词为假。个体的取值范围为个体域。

- 谓词中可以有若干个个体变量，如father(x, y) 表示x是y父亲。
  $P(x)$是一元谓词（包含一个个体），
  $P(x_1, x_2, \ldots, x_n)$被称为n元谓词（包含若干个体）。

量词

- 全称量词(universalquantifier, ∀)

  全称量词用符号∀表示，表示一切的、凡是的、所有的、 每一个等。$\forall x$表示定义域中的所有个体，$\forall xP(x)$表示定 义域中的所有个体具有性质P

- 存在量词(existential quantifier, ∃)

  存在量词用符号∃表示，表示存在、有一个、某些等。 $\exists x$表示定义域中存在一个或若干个个体，$\exists xP(x)$表示定 义域中存在一个个体或若干个体具有性质P

全称量词和存在量词出现的次序将影响命题的意思。

- $(\forall x)(\exists y)(Employee(x) \rightarrow Manager(y, x))$:

  每个雇员都有一个经理。

- $(\exists y)(\forall x)(Employee(x) \rightarrow Manager(y, x))$:

  有一个人是所有雇员的经理。

- 全称量词消去(Universal Instantiation, UI): $(\forall x)A\ x \rightarrow A(y)$
- 全称量词引入(Universal Generalization, UG): $A(y) \rightarrow (\forall x)A\ (x)$
- 存在量词消去(Existential Instantiation, EI): $(\exists x)A(x) \rightarrow A(c)$
- 存在量词引入(Existential Generalization, EG): $A(c) \rightarrow (\exists x)A(x)$

函词

- 函词中个体变元用个体常量（来自定义域）代入 后结果仍是个体（值域）， 如定义函数$f\ x$ =x+10，则f 2 =12
- 谓词中个体变元用个体常量带入后就变成了命题， 如C$ar\ x$ (x是车)这个谓词中x用吉普车代替， 则Car 吉普车 是命题。
- **函词是从定义域到值域的映射； 谓词是从定义域到{$True$,$False$}的映射**

一阶逻辑转化成合取范式 (CNF)

一阶逻辑的推理： 转化成合取范式 (CNF)

一阶逻辑的推理：转化成合取范式(CNF)

- “everyone whoe loves all animals is loved by someone”

$\forall x\ [\forall y\ Animal(y) \Rightarrow Loves(x,y)] \Rightarrow [\exists y\ Loves(y,x)]$

- 消除蕴含词

$\forall x\ [\neg \forall y\ \neg Animal(y) \vee Loves(x,y)] \vee [\exists y\ Loves(y,x)]$

- 将否定符号内移

$\neg \forall x\ p \equiv \exists x\ \neg p,$
$\neg \exists x\ p \equiv \forall x\ \neg p$

$\forall x\ [\exists y\ \neg(\neg Animal(y) \vee Loves(x,y))] \vee [\exists y\ Loves(y,x)]$
$\forall x\ [\exists y\ \neg\neg Animal(y) \wedge \neg Loves(x,y)] \vee [\exists y\ Loves(y,x)]$
$\forall x\ [\exists y\ Animal(y) \wedge \neg Loves(x,y)] \vee [\exists y\ Loves(y,x)]$

一阶逻辑的推理：转化成合取范式(CNF)

- 变量标准化：每一个量词应该使用不同的符号

$\forall x\ [\exists y\ Animal(y) \wedge \neg Loves(x,y)] \vee [\exists z\ Loves(z,x)]$

- Skolem化：消除存在量词

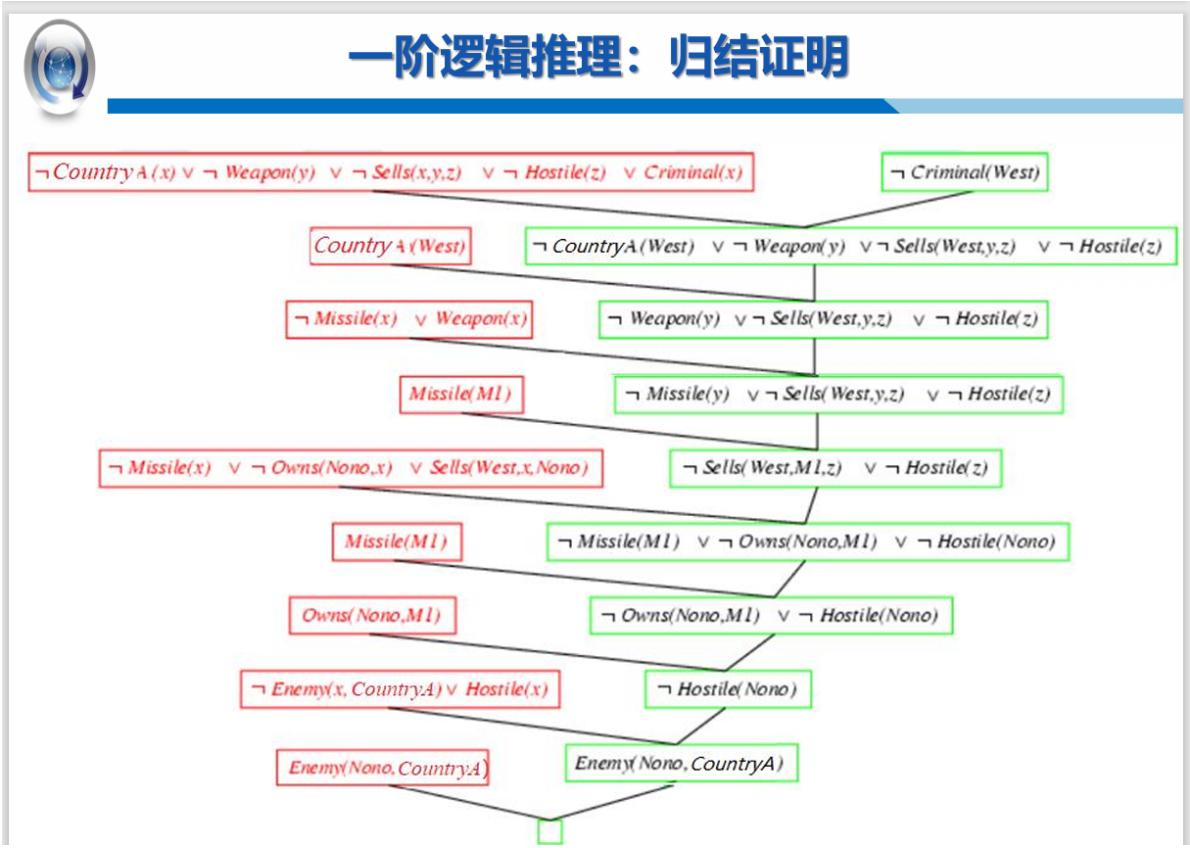
$\forall x\ [Animal(F(x)) \wedge \neg Loves(x,F(x))] \vee Loves(G(x),x)$

- 删除全称量词

$[Animal(F(x)) \wedge \neg Loves(x,F(x))] \vee Loves(G(x),x)$

- 将$\wedge$分配到$\vee$中

$[Animal(F(x)) \vee Loves(G(x),x)] \wedge [\neg Loves(x,F(x)) \vee Loves(G(x),x)]$



第十讲 不确定性的量化

不确定性概述

Agent的环境可能是部分可观察的或者不确定的，也可能是部分可观察并且不确定的

怎样处理部分可观察和不确定性问题?

当解释观察到的部分信息时，逻辑Agent必须考虑每一种逻辑上可能的解释——信念状态

- 信念状态需要考虑很多种可能性，导致问题表示庞大而复杂

逻辑Agent和概率Agent认识约束不同:

- 逻辑Agent相信每个语句是正确的或者错误的
- 概率Agent为每个语句赋予一个0到1之间的数值 作为其信念度

决策理论=概率理论+效用理论

决策理论的基本思想:

- 一个Agent是理性的，当且仅当它选择能产生最高期望效用的行动，称为期望效用最大化
- 期望：统计学的平均

逻辑断言考虑的是要排除所有那些断言不 成立的世界。

概率断言考虑的是各种可能世界的可能性 有多大。

基本概率符号

- 样本空间(Ω，omega的大写)： 概率理论中定义所有可能世界组成的集合
- ω（omega的小写）表示样本空间中的一个 样本，即ω是一个特定的可能世界
- 概率理论的基本公理规定，每个可能世界具 有一个0到1之间的概率，且样本空间中的可能 世界的总概率是1。

- 概率模型： 样本空间 Ω 为每一个可能世界附 一个数值概率 p(ω)
- 事件 P (w) (event):样本空间中的可能世界集合
- 基本元素：随机变量
  - 大写字母开头 e.g.Total:掷两个骰子的点数
- 定义域：变量所有可能值组成的集合
  - A=true简写成a,A=false简写成¬a
- 原子命题通过对随机变量赋值得到的命题， 比如Weather = sunny, Cavity = false
- 复杂命题指原子命题通过连接符号连接而成 的命题Weather = sunny ∨ Cavity = false
- 讨论随机变量Weather每个可能取值的概率：
  - P(Weather = sunny) = 0.6
  - P(Weather = rain) = 0.1
  - P(Weather = cloudy) = 0.29
  - P(Weather = snow) = 0.01

  =》可以简写为： P(Weather) = <0.6, 0.1, 0.29, 0.01> ， P定义了随机变量Weather的一个概率分布
- 概率密度函数是用来描述概率分布的函数
- 先验概率： 又称无条件概率
- 概率分布： 所有可能的随机变量赋予概率值，所有可能事件的概率值之和为1
- 联合概率分布： 对多个随机变量中的每个变 量发生的可能性进行概率赋值
- 条件概率： 又称为后验概率
- 条件概率的定义：P(a | b) = P(a ∧ b) / P(b) if P(b) > 0
  - 运用乘法规则我们得到另一种表示形式 **P(a ∧ b) = P(a | b) P(b) = P(b | a) P(a)**

概率推理

根据已观察到的证据计算查询命题的后验概 率。=》使用完全联合概率分布作为“知识库”，从中可以导出所有问题的答案。

条件独立性

命题a和b之间的独立性<=>P (b | a) = P (b)<=> P (a | b) = P (a)<=> P (a ^ b) = P (a) P (b)

- 应用条件独立性能够将完全联合分布的公式从指数级的项数降到接近线性级
- 条件独立性是依赖于我们从环境中的知识得到

$$P(cause|effect)=\frac{P(effect|cause)P(cause)}{P(effect)}$$

贝叶斯规则

联合概率规则 P(a∧b) = P(a | b) P(b) = P(b | a) P(a)

贝叶斯规则 P(a | b) = (P(b | a) P(a)) / P(b)

应用贝叶斯规则 将未知因素cause造成的结果effect看作是证据， 而确定那个未知因素cause。这种情况下，贝叶斯规则变成

$$P(cause|effect)=\frac{P(effect|cause)P(cause)}{P(effect)}$$

第十一讲 贝叶斯网络

条件独立性

给定Cavity，Catch是Toothache的条件独立 : P(Catch | Toothache, Cavity) = P(Catch | Cavity)

=》**注意到就是上述式子有没有Toothache没差别**

变量之间的独立性和条件独立性可以大大减 少完全联合概率分布所需要的概率数目



- 无条件的（绝对）独立非常罕见
- 条件独立性是我们关于不确定环境中的知识最基本的、鲁棒性形式
- 对于给定Z，X是Y的条件独立性 $X \perp\!\!\!\perp Y|Z$
- 当且仅当：$\forall x,y,z : P(x,y|z) = P(x|z)P(y|z)$
- 或者，等价地，当且仅当：$\forall x,y,z : P(x|z,y) = P(x|z)$

贝叶斯网络

贝叶斯网络：一种简单的用于表示变量之间 条件独立性的有向无环图

- 结点对应于随机变量
- 有向边连接结点对
- 每个结点都有一个条件概率分布

确定性结点：子结点由父结点决定，没有 任何不确定性。

不确定性结点：子结点与父结点的因果关系 不确定，存在概率表示。

单联通网络：任意两个结点之间顶多只有一条无向 路径

- 精确推理的时间和空间复杂度跟网络规模呈线性关系

多联通网络：非单联通

贝叶斯网络的精确推理

枚举算吧~，详情可见作业

贝叶斯网络的近似推理

采样算法：根据已知概率分布生成样本

最简单的随机方法

- 不考虑证据变量值
- 按拓扑结构对变量顺序采样
- 子结点样本依赖于父结点样 本的赋值

第十二讲 时间上的概率推理

时间和不确定性

静态世界中随机变量有唯一固定的取值

动态世界中随机变量的取值随时间而变

状态的改变过程可以被视为一系列快照，或称为时间片（time slice）

转移模型与传感器模型

- 给定问题的状态变量和证据变量的集合
- 转移模型：描述在给定过去状态变量的值之后，确定最新状态 变量的概率分布P($Xt|X0{:}t-1$)
- 随着时间t的增长，集合$X0{:}t-1$的大小没有约束
- 马尔可夫假设（Markov assumption ）
  - 当前状态只依赖于有限的固定数量的过去状态
  - 马尔可夫过程（Markov process）
  - 马尔科夫链（Markov chain）
- 一阶马尔可夫过程：当前状态只依赖于前一个状态
- 二阶马尔可夫过程：当前状态只依赖于前两个状态
- 传感器模型（观察模型）：证据变量Et可能依赖于前面的变量， 也依赖于当前的状态变量
- 传感器马尔可夫假设:当前的状态产生当前的传感器值 P($Et|X0{:}t,E0{:}t-1$)= P($Et|Xt$)
- 稳态过程：转移模型和传感器模型对所有的时间片t都是相同的，即P($Xt|Xt-1$)，P($Et|Xt$)对所有的t都是相同的。

马尔可夫模型中的推理

平稳性假设:转移概率相同

马尔可夫模型概述

- 对所有t的明确的假设： $X_t \perp\!\!\!\perp X_1, \ldots, X_{t-2} \mid X_{t-1}$
- 因此，联合分布可以写成：

$$P(X_1, X_2, \ldots, X_T) = P(X_1)P(X_2|X_1)P(X_3|X_2)\ldots P(X_T|X_{T-1})$$
$$= P(X_1)\prod_{t=2}^{T} P(X_t|X_{t-1})$$

- 隐含条件独立性：(试图证明这个!)
  - 考虑到目前过去变量独立于未来变量
  - i.e.， 如果 $t_1 < t_2 < t_3$ 或 $t_1 > t_2 > t_3$ 那么：$X_{t_1} \perp\!\!\!\perp X_{t_3} \mid X_{t_2}$
- 附加平稳假设：$P(X_t \mid X_{t-1})$ 是对所有t相同的

隐马尔可夫模型

- 对于大多数问题马尔可夫链不太有用
- 需要观察来更新你的信念
- 隐马尔可夫模型(Hidden Markov Model，HMM)
  - 用单个随机变量描述过程状态的时序概率
  - 潜在的马尔可夫链状态X
  - 观察在每个步的输出
  - 模型有多个变量，可以将多个变量组合成单个“大变量”

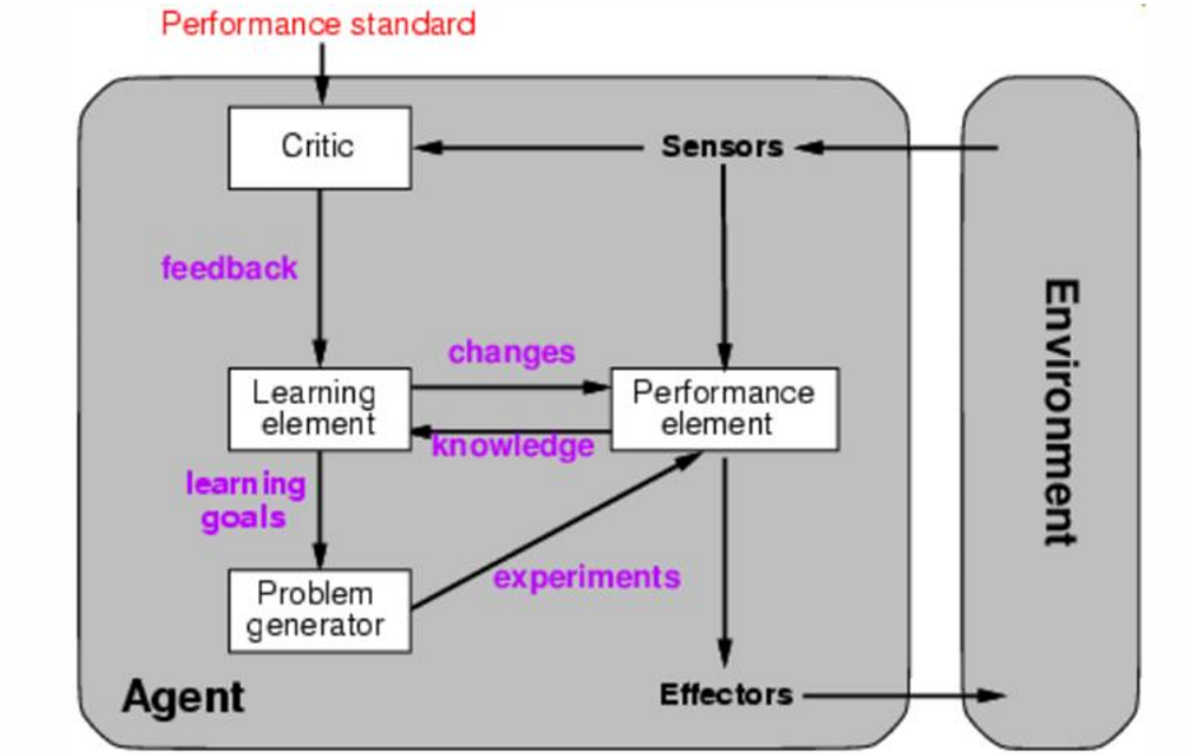
在隐马尔可夫模型中，我们 不仅仅有这么一串可见状态链，还有一串隐含状态链。

貌似另外个班就没讲

第十三章 机器学习

机器学习概述

学习Agent



机器学习：机器可以自动“学习”的算法，即从数据中自动分析获得规律，并利用规律对未知数据进行预测的算法。

机器学习的分类

半监督学习 (semi-supervised learning):

- 监督学习(supervised learning) 数据有标签、一般为回归或分类等任务
  - 在已知输入和输出 的情况下, 通过训练建立起输入到输 出的映射模型
  - 应用最广的机器学习方法, 可进一步分为分类和回归两类算法
    - 分类问题的主要目的是预测分类标签，标签来 自预定义的可选列表。分类问题可分为二分类 和多分类。
    - 回归任务的目标是预测一个连续值。
  - 重要元素：标注数据、学习模型、损失函数
  - 监督学习方法又可以分为生成方法(generative approach)和判别方法(discriminative approach) 。所学到的模型分别称为生成模型(generative model)和判别模型 (discriminative model)
    - 生成模型试图对观测数据和标签之间的联合概率分布进行建模，即 $P(X,Y)P(X,Y)$，其中 XX 是观测数据（输入），YY 是标签（输出）。
    - 生成模型可以通过对联合概率分布进行建模来学习数据的整体结构和特征。它们不仅可以生成新的数据样本，还可以用于预测，并且可以处理缺失数据等任务。
    - 常见的生成模型包括朴素贝叶斯（Naive Bayes）、隐马尔可夫模型（Hidden Markov Models，HMM）、生成对抗网络（Generative Adversarial Networks，GAN）等。
    - 判别模型则直接对条件概率分布 $P(Y|X)P(Y|X)$ 进行建模，即给定观测数据 XX，预测其对应的标签 YY 的概率。

        - 判别模型关注于数据的决策边界或分类面，它们不会显式地建模数据的生成过程，而是专注于学习如何将输入数据映射到相应的输出。
        - 判别模型通常用于分类、回归等任务，它们更加直接，通常在性能上有更好的表现。
        - 常见的判别模型包括逻辑回归（Logistic Regression）、支持向量机（Support Vector Machines，SVM）、决策树（Decision Trees）等。

- 无监督学习(un-supervised learning) 数据无标签、一般为聚类或若干降维任务
    - 学习过程中, 只有输入数据, 没有对应的输出数据作为 参考
    - 通过模型自我归纳, 自我改进来获得数据中隐含的结构 信息
    - 优点：不需要进行繁重的人工数据标注
    - 弱监督学习(Weakly Supervised Learning)：在标注数据不完全、不确切或不精确等场景下构建模 型

强化学习(reinforcement learning) 序列数据决策学习，一般为与从环境交互中学习

- Agent选择一个动作用于环境，环境接受该动作 后状态发生变化，同时产生一个强化信号(奖或惩)反馈给Agent，Agent根据强化信号和环境当 前状态再选择下一个动作，选择的原则是使受到正强化(奖)的概率增大。

机器学习的其他概念

训练集：用于训练目标函数的数据集合

测试集：用于测试目标函数好坏的数据集合

k-折交叉验证思想：每个样本既作为训练数据，又作为 测试数据

经验风险(empirical risk)：训练集中数据产生的损 失。经验风险越小说明 学习模型对训练数据拟 合程度越好。

期望风险(expected risk)：当测试集中存在无穷多数 据时产生的损失。期望风 险越小，学习所得模型越好。

过拟合：拟合模型的时候，过分关注训练集的细节，训练集学习得到的目标函数在训练集上性能很好，在测试集上性能不高。训练出的模型太复杂，不能泛化到新数据上的 模型，就被称为过拟合：

- Early stopping（适当的stopping criterion） Early stopping便是一种迭代次数截断的方法来防止过拟 合的方法，即在模型对训练数据集迭代收敛之前停止迭代 来防止过拟合
- 数据集扩增 数据集扩增即需要得到更多的符合要求的数据，即和已有的 数据是独立同分布的，或者近似独立同分布的。一般方法有： 从数据源头采集更多数据，复制原有数据并加上随机噪声，重采样，根据当前数据集估计数据分布参数，使用该分布产 生更多数据等。 □如果避免过拟合?
- 正则化方法 一般有L1正则与L2正则等（就是在成本函数上加参数的平方和）
- Dropout 正则是通过在代价函数后面加上正则项来防止模型过拟 合的。而在神经网络中，有一种方法是通过修改神经网络 本身结构来实现的，其名为Dropout。（就是随机删掉几个神经元，解释来自康康）

欠拟合相反，如果模型过于简单，那么就可能无法抓住 数据的全部内容以及数据中的变化。甚至可能模型在训练集上的表现也很差。

归纳学习

机器学习按推理方式分类：

- 基于演绎的学习（解释学习）。
- 基于归纳的学习 （示例学习、发现学习等 ）。

什么是归纳？ =》基于过去的事实预测将来的发生的事情

构造/调整假说h与函数f在训练集上一致

h是一致的如果h与f在所有的训练样本上都相同

- 当存在多个h与f在训练样本上一致如何选择？ 奥坎姆剃刀:选择与数据一致的最简单假设

K临近算法

K近邻算法(K-Nearest Neighbors, KNN) K近邻算法是一种常用的机器学习分类方法。

算法的核心思想：物以类聚

算法的原理是：给定一个待分类样本, 通过统计离它最近的前k个邻居中包含 样本最多的类别，作为待分类样本的类别。

核心要素：

- 距离度量准则：常用欧式距离和曼哈顿距离
- K的大小

朴素贝叶斯算法

基于贝叶斯规则和条件独立假设的分类算法。

通过计算给定样本特征对各个类别的条件概率来预测 分类，选择条件概率最大项对应的类别作为样本的预测类别。



f_1	f_2	f_3	f_4	y
0	1	1	0	**1**
0	0	1	1	**1**
1	0	1	0	**1**
0	0	1	1	**1**
0	0	0	0	**1**
1	0	0	1	**0**
1	1	0	1	**0**
1	0	0	0	**0**
1	1	0	1	**0**
1	0	1	1	**0**

$R_1(1,1)=1/5$：所有$f_1=1$的正样本占所有正样本的比例；
$R_1(0,1)=4/5$：所有$f_1=0$的正样本占所有正样本的比例；
$R_1(1,0)=5/5$：所有$f_1=1$的负样本占所有负样本的比例；
$R_1(0,0)=0/5$：所有$f_1=0$的负样本占所有负样本的比例；

决策树算法

决策树由节点和分支组成，每一条从根结点（对 分类结果贡献最大的属性）到叶子结点（分类结 果）的路径都代表一条决策的规则。

决策树分支策略：

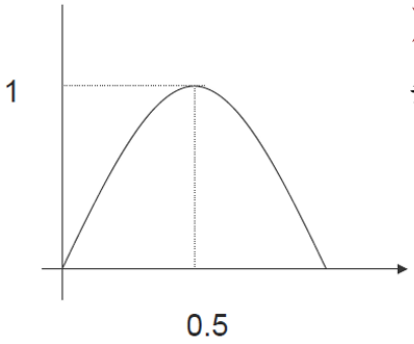
- 如果剩余样本都是正样本或者都是负样本，表 示树分裂结束，回答Yes或No
- 如果剩余样本包含正样本和负样本，继续分裂 属性
- 如果没有剩余样本，则返回缺省值
- 如果剩余样本包含正样本和负样本但没有属性 可以分裂，意味着出现错误，或者领域是非确 定性的，或者没有观察到区分样本的属性。则 此最好的办法是返回剩余样本中得票最多的分 类

信息理论：熵

➢ 熵H(x)：衡量数据集的纯度

$$H(V) = -\sum_k P(v_k)\log_2 P(v_k)$$

➢ 熵曲线



➢ e. g. 两类样本(P, N)，p个正样本数，n个负样本，p=9, n=5

$$H([9,5]) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.94$$

第十四讲 人工神经网络

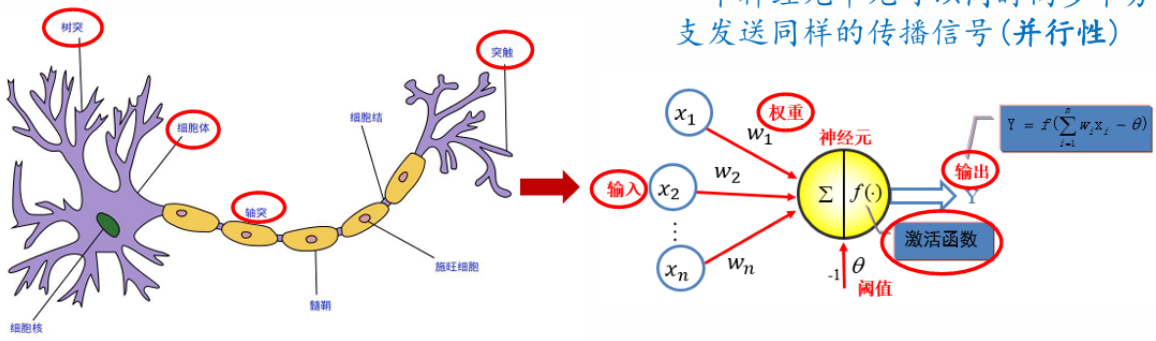
人工神经网络ANN概述

➢ **人工神经网络(Artificial Neural Network, ANN)**

人工神经网络模拟人的大脑，人脑由大量的神经细胞构成，称为神经元。

神经元单元由许多带权值的有向链相连接(**轴突**)

一个神经元单元可以同时向多个分支发送同样的传播信号(**并行性**)



人脑神经元结构　　人工神经元模型

Biological Neural Network			*Artificial Neural Network*	
Soma	细胞体	→	Neuron	神经元
Dendrite	树突	→	Input	输入
Axon	轴突	→	Output	输出
Synapse	突触	→	Weight	权重

算法的原理是：

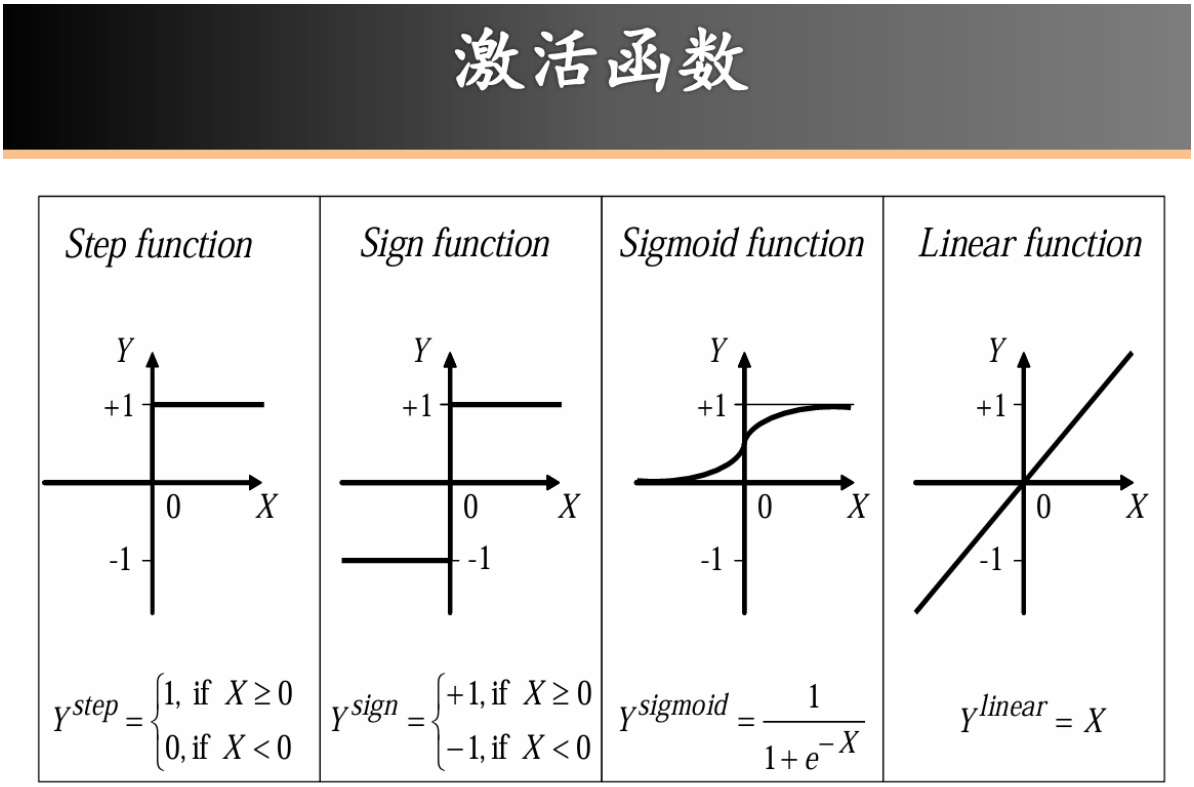
将训练数据输入到神经网络

计算真实输出与模型输出之间的误差

通过调节权重来减小误差

学习出神经网络模型的参数

几个常用激活函数：



感知器算法

感知器算法主要用于二分类问题的线性可分模型。

算法原理是：求出将训练数据进行线性划分的分离超平面

感知器算法的目标是通过最小化损失函数， 学习出模型中的参数w和b，从而找到将样 本线性划分的分离超平面。

算法步骤：

- 第一步：初始化：设定初始的权值w1,w2,...,wn和阈值□为 [-0.5,0.5]之间的随机数
- 第二步：计算激活函数值：根据输入x1(p), x2(p),..., xn(p) 和权值 w1,w2,...,wn计算输出Y(p)，其中p表示 迭代的轮数
- 第三步：更新权值
- 第四步：迭代循环：增加p值，不断重复步骤二和步骤三直到 收敛

多层人工神经网络

一个输入层，至少一个中间隐藏层，一 个输出层

反馈的神经网络

信号从输入层向前传播，误差从输出层 向后反馈

隐藏层：

- 行为不能被外界观察到
- 商业的人工神经网络集成了3-4层隐藏层， 每一层包含10-1000个神经元
- 实验中的人工神经网络甚至包含上百万 个神经元

算法思想：

- 训练数据输入到神经网络中，神经网络计算 真实输出与实际输出之间的差异，然后通过 调节权值来减小这种差异
- 两阶段
    - 训练数据输入，通过前向层计算到输出层输 出结果
    - 计算真实输出与实际输出之间的错误，通过 反向从输出层-隐藏层-输入层的调节权值来 减少错误